IAO Architecture Pipeline

Python-based automation pipeline for generating **TOGAF BDAT-aligned** Solution Architecture Documents (SADs) across the IDM 2.0 (SAP S/4HANA transformation) program.

Pulls live data from enterprise APIs (Smartsheet, IAPM, SAP BIC, SAP S/4HANA OData), enriches it, and produces per-capability architecture documents — Business, Data, Application, and Technology architecture — for each tower.

Towers

Tower	Description
FPR	Forecast, Planning & Replenishment
OTC-IF	Order to Cash — Intel Foundry
OTC-IP	Order to Cash — Intel Products
FTS-IF	Forecast to Stock — Intel Foundry
FTS-IP	Forecast to Stock — Intel Products
PTP	Procure to Pay
MDM	Master Data Management
E2E	End-to-End (cross-tower)

Project Structure

```
├── README.md                   # This file
├── requirements.txt            # Python dependencies
├── .env.example                # Credential template (copy to .env)
├── .gitignore
│
├── scripts/                    # Pipeline CLI entry points
│   ├── update_sad_from_smartsheet.py
│   ├── gen_pdf.py
│   ├── gen_xlsx_templates.py
│   ├── scaffold_inputs.py
│   ├── backfill_bpmn_to_xlsx.py
│   ├── backfill_sample_data.py
│   ├── cleanup_csvs.py
│   └── sync_sharepoint.py
│
├── src/                        # Reusable Python library modules
│   ├── smartsheet_loader.py    #   Smartsheet data loader
│   ├── bpmn_parser.py          #   BPMN XML parser
│   ├── csv_parser.py           #   CSV parsing utilities
│   ├── xlsx_loader.py          #   Excel workbook loader
│   ├── mermaid_builder.py      #   Mermaid diagram generator
│   ├── gen_systems_arch.py     #   Systems architecture doc generator
│   ├── diff_engine.py          #   Document diff engine
│   ├── context_loader.py       #   Tower/capability context loader
│   ├── iapm_lookup.py          #   IAPM application lookup
│   └── config.py               #   Configuration management
│
├── mcp_servers/                # MCP tool servers for Copilot integration
│   ├── smartsheet_server.py    #   Smartsheet API tools (live + CSV fallback)
│   ├── iapm_server.py          #   IAPM application portfolio tools
│   ├── jira_server.py          #   JIRA integration tools (placeholder)
│   ├── sap_odata_server.py     #   SAP OData tools (placeholder)
│   └── bic_server.py           #   SAP BIC process design tools (placeholder)
│
├── templates/                  # Jinja2 templates for doc generation
│   ├── capability_architecture.md.j2
│   ├── systems_architecture.md.j2
│   ├── CurrentFlows_TEMPLATE.xlsx
│   └── FutureFlows_TEMPLATE.xlsx
│
├── towers/                     # Tower data (one subfolder per tower)
│   ├── FPR/                    #   See "Capability Folder Structure" below
│   ├── OTC-IF/
│   ├── OTC-IP/
│   ├── FTS-IF/
│   ├── FTS-IP/
```

```
│   ├── PTP/
│   ├── MDM/
│   └── E2E/
│
```

Capability Folder Structure

Each capability under a tower follows this layout:

```
towers/<TOWER>/<Process Group>/<CAP-ID>/
├── input/
│   └── data/                              # ◀ Architect-editable Excel inputs
│       ├── CurrentFlows.xlsx               #   Current-state process flows
│       ├── FutureFlows.xlsx                #   Future-state process flows
│       ├── R3_CurrentFlows.xlsx            #   Release 3 current-state flows
│       └── R3_FutureFlows.xlsx             #   Release 3 future-state flows
└── output/
    └── docs/
        └── systems-architecture/
            ├── <CAP-ID>-Architecture.html  # ◀ Generated HTML (synced to SharePoint)
            ├── <CAP-ID>-Architecture.pdf   # ◀ Generated PDF (synced to SharePoint)
            ├── <CAP-ID>-Architecture.md    #   Generated Markdown (GitHub only)
            └── svg/                        #   Rendered Mermaid diagrams
```

Example (FPR tower, DS-020 capability):

```
towers/FPR/DS Provide Decision Support/DS-020/
├── input/data/
│   ├── CurrentFlows.xlsx
│   ├── FutureFlows.xlsx
│   ├── R3_CurrentFlows.xlsx
│   └── R3_FutureFlows.xlsx
└── output/docs/systems-architecture/
    ├── DS-020-Architecture.html
    ├── DS-020-Architecture.pdf
    └── DS-020-Architecture.md
```

> **What gets synced to SharePoint?** Only **HTML, PDF, and SVG** files from `output/`. Markdown files remain in GitHub only.
>
> **What do architects edit?** The **`.xlsx` files** in `input/data/`. These drive the Business, Data, Application, and Technology architecture sections.

```
├── data/                       # Cached/extracted data
│   ├── smartsheet/             #   Smartsheet CSV exports
│   ├── iapm/                   #   IAPM application data
│   ├── bic/                    #   BIC process model data
│   ├── sap_odata/              #   SAP OData extractions
│   ├── enriched/               #   Enriched flow data
│   ├── mermaid/                #   Generated Mermaid diagrams
│   └── xref/                   #   Cross-reference mappings
│
├── docs/                       # Planning & architecture documentation
├── notebooks/                  # Jupyter notebooks (future)
├── config/                     # Configuration files
├── .github/workflows/          # GitHub Actions (scheduled generation)
├── reference/                  # Reference scripts & audit data
└── _archive/                   # Archived exploration/probe scripts
```

Pipeline Scripts

All scripts live in `scripts/` and are run from the project root:

Script	Purpose
`scripts/update_sad_from_smartsheet.py`	Update all SADs with live Smartsheet data (roadmap, RICEFW status, RAID log)
`scripts/gen_pdf.py`	Convert architecture Markdown to styled HTML and PDF
`scripts/gen_xlsx_templates.py`	Generate multi-tab Excel input templates for tower architects
`scripts/scaffold_inputs.py`	Deploy input templates and upgrade flow CSVs across all capabilities
`scripts/backfill_bpmn_to_xlsx.py`	Populate Business Architecture tabs from BPMN XML exports
`scripts/backfill_sample_data.py`	Pre-fill supplementary tabs with contextually grounded sample data
`scripts/cleanup_csvs.py`	Remove legacy CSV inputs superseded by xlsx workbooks
`scripts/sync_sharepoint.py`	Upload generated architecture docs to SharePoint Online

Setup

Prerequisites

- Python 3.12+
- Access to Smartsheet API (token)
- Optional: IAPM, SAP BIC, SAP S/4HANA credentials for full pipeline

Installation

```
# Clone the repository
git clone <repo-url>
cd IAO-JPNotebookPython

# Create virtual environment
python -m venv .venv

# Activate (Windows)
.venv\Scripts\Activate.ps1

# Activate (Linux/macOS)
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

Configuration

```
# Copy the environment template
cp .env.example .env

# Edit .env and add your credentials
# At minimum, set SMARTSHEET_TOKEN for the core pipeline
```

See .env.example for all available configuration variables.

Usage

Generate / Update Architecture Documents

```
# Update all tower SADs with live Smartsheet data
python scripts/update_sad_from_smartsheet.py

# Update a specific tower
python scripts/update_sad_from_smartsheet.py --tower FPR

# Preview changes without writing
python scripts/update_sad_from_smartsheet.py --dry-run
```

Generate Output Formats

```
# Generate HTML + PDF for all towers
python scripts/gen_pdf.py

# Generate HTML only for a specific tower
python scripts/gen_pdf.py --tower FPR --html-only
```

Scaffold New Capabilities

```
# Deploy input templates to all capabilities
python scripts/gen_xlsx_templates.py --deploy

# Backfill BPMN data into workbooks
python scripts/backfill_bpmn_to_xlsx.py --tower FPR
```

MCP Servers & API Access

The `mcp_servers/` directory contains Model Context Protocol servers that expose enterprise data as tools for GitHub Copilot. These are registered in `.vscode/settings.json` and run as stdio subprocesses.

Why API Access Matters

Direct API access is critical for producing accurate, up-to-date architecture documents. Without it, the pipeline relies on stale CSV/Excel exports that drift out of sync with source systems. Each API enables a specific layer of automation:

API	Architecture Impact	Without It
Smartsheet	Live RICEFW status, RAID items, delivery timelines per capability	Manual CSV export, data goes stale within days
IAPM	Application portfolio lookup — hosting, lifecycle, ownership for Application Architecture	Offline CSV (30K apps) — works but may miss recent changes
JIRA	Test case readiness, defect counts, sprint status for Build Readiness sections	No automated QA/test visibility
SAP OData	Dev object inventory, transport status, CDS views for Technology Architecture	Manual collection from SAP GUI
SAP BIC	BPMN process models — swim lanes, tasks, gateways for Business Architecture	Manual BPMN XML export from BIC UI

Server Status

Server	MCP Name	Status	Tools	Required Credentials
Smartsheet	`iao-smartsheet`	**Working**	`get_ricefw_objects` , `get_raid_items` , `get_timeline` , `get_tower_summary` , `read_sheet_live` , `list_known_sheets`	`SMARTSHEET_TOKEN` (falls back to CSV cache if missing)
IAPM	`iao-iapm`	**Working**	`lookup_application` , `search_applications` , `get_app_status`	None — reads from `data/iapm/IAPM_All_Solutions.csv`
JIRA	`iao-jira`	Placeholder	`get_test_cases` , `get_defects` , `get_sprint_status` , `search_issues`	`JIRA_BASE_URL` , `JIRA_USER_EMAIL` , `JIRA_API_TOKEN`
SAP OData	`iao-sap-odata`	Placeholder	`get_dev_objects` , `get_transport_status` , `get_cds_views` , `get_fiori_apps`	`SAP_BI0_HOST` , `SAP_BI0_USER` , `SAP_BI0_PASS` (+ `S_SERVICE` SAP role)
SAP BIC	`iao-bic`	Placeholder	`get_process_models` , `get_process_detail` , `search_processes` , `export_bpmn`	`BIC_AUTH_TOKEN` (browser extraction; expires frequently)

Setting Up MCP Servers

The servers are pre-registered in `.vscode/settings.json` . To activate them:

1. **Install the MCP SDK** (included in `requirements.txt`):

   ```
   pip install -r requirements.txt
   ```

2. **Add credentials** to `.env` for each API you have access to (see `.env.example`)
3. **Restart VS Code** — Copilot auto-discovers the servers via settings.json
4. **Verify** by asking Copilot: *"List the available Smartsheet tools"* or *"Search IAPM for SAP"*

Requesting API Access

For Intel enterprise systems, API access requires formal provisioning:

System	Access Method	What to Request
Smartsheet	API token from Smartsheet admin	Bearer token with read access to IAO workspace sheets
IAPM	Azure AD Bearer token	Access to the IAPM application portfolio (auto-granted for most Intel employees)
JIRA	Personal Access Token	Create at JIRA profile → Personal Access Tokens; needs read access to IDM 2.0 projects
SAP S/4HANA	SAP Gateway access	Request `S_SERVICE` authorization role for OData catalog on BI0/DI0 systems via SAP access request form
SAP BIC	Session token (browser)	Access to BIC process design portal; token extracted from browser DevTools (F12 → Network → Authorization header)

CI/CD Workflows

Three GitHub Actions workflows automate the full document lifecycle:

Workflow	Trigger	Purpose
`generate-architecture.yml`	Weekly (Mon 06:00 UTC), push to main, manual	Regenerate MDs, update from Smartsheet, generate HTML, sync to SharePoint
`sharepoint-reverse-sync.yml`	Power Automate webhook, manual	Pull updated Excel inputs from SharePoint → commit → triggers forward generation
`deploy-pages.yml`	After `Generate Architecture Docs` completes, manual	Deploy HTML outputs to GitHub Pages for browser viewing

Pipeline Flow

```
SharePoint (architect edits xlsx)
       │
       ▼
Power Automate → repository_dispatch
       │
       ▼
sharepoint-reverse-sync.yml
  ├── Download updated file from SharePoint via Graph API
  ├── Commit to towers/<TOWER>/<CAP>/input/data/
  └── Push to main  ──────────────────────┐
                                          ▼
                                generate-architecture.yml
                                  ├── Generate architecture MDs
                                  ├── Update SADs from Smartsheet
                                  ├── Generate HTML outputs
                                  ├── Commit outputs to repo
                                  ├── Sync HTML/PDF/SVG to SharePoint
                                  └── Trigger deploy-pages.yml
                                          │
                                          ▼
                                      deploy-pages.yml
                                        └── Deploy HTML to GitHub Pages
```

SharePoint Two-Way Sync

The pipeline supports two-way sync between GitHub and SharePoint so that tower architects can update Excel input files directly on SharePoint without needing Git access.

How It Works

1. **Forward sync** (GitHub → SharePoint): After docs are generated, `sync_sharepoint.py` uploads **HTML, PDF, and SVG** files to the SharePoint document library. Markdown files stay in GitHub only.
2. **Reverse sync** (SharePoint → GitHub): When an architect updates an Excel input file (CurrentFlows.xlsx, FutureFlows.xlsx, etc.) on SharePoint, a Power Automate flow triggers `sharepoint-reverse-sync.yml` to pull the file into the repo and regenerate docs.

Folder Structure on SharePoint

The sync maps the GitHub `towers/` structure to SharePoint:

```
SharePoint: Shared Documents / Architecture/SAD /
├── FPR/
│   └── docs/systems-architecture/
│       ├── DS-020-Architecture.html    ◀ Viewable in browser / downloadable
│       ├── DS-020-Architecture.pdf     ◀ Downloadable
│       └── svg/
│           └── *.svg
├── OTC-IF/
├── PTP/
└── ...
```

SharePoint HTML viewing: Architects can click an HTML file to open it in the browser preview, or download it locally. If the browser preview strips JavaScript (preventing Mermaid rendering), the PDF version is the reliable fallback.

Default mapping (from `.env`):

- `SP_DOC_LIBRARY = Shared Documents`
- `SP_TARGET_FOLDER = Architecture/SAD`

Syncing Specific Towers

To sync only certain towers instead of all:

```
# Sync a single tower to SharePoint
python scripts/sync_sharepoint.py --tower FPR

# Sync all towers
python scripts/sync_sharepoint.py --all
```

In the GitHub Actions manual dispatch, enter a tower shortcode to limit the scope:

> **Actions → Generate Architecture Docs → Run workflow → Tower shortcode**: `FPR`

Setting Up Reverse Sync (SharePoint → GitHub)

Prerequisites

1. **Azure AD App Registration** with `Sites.ReadWrite.All` Graph API permission
2. **GitHub Personal Access Token** (PAT) with `repo` scope for Power Automate to call the dispatch API
3. **Power Automate** (or Logic App) flow on the SharePoint site

Step 1: Configure GitHub Secrets

Go to **Settings → Secrets and variables → Actions** and add:

Secret	Description
`SMARTSHEET_TOKEN`	Smartsheet API token
`SP_TENANT_ID`	Azure AD tenant ID
`SP_CLIENT_ID`	Azure AD app client ID
`SP_CLIENT_SECRET`	Azure AD app client secret
`SP_SITE_URL`	SharePoint site URL (e.g., `https://intel.sharepoint.com/sites/IAO-Architecture`)
`SP_DOC_LIBRARY`	Document library name (default: `Shared Documents`)
`SP_TARGET_FOLDER`	Target folder path (default: `Architecture/SAD`)

Optional (for full pipeline): `IAPM_BEARER_TOKEN` , `BIC_AUTH_TOKEN`

Step 2: Create Power Automate Flow

Create a flow that fires when an `.xlsx` file changes in the SharePoint document library:

1. **Trigger**: *When a file is modified (properties only)* in the `Architecture/SAD` folder
2. **Filter**: File extension = `.xlsx`
3. **Parse path**: Extract tower and capability from the file path
   - Example: `Architecture/SAD/FPR/DS-001/input/data/CurrentFlows.xlsx`
   - Tower = `FPR` , Capability = `DS-001` , File = `CurrentFlows.xlsx`
4. **HTTP action**: Send `repository_dispatch` to GitHub:

```
POST https://api.github.com/repos/<OWNER>/<REPO>/dispatches
Headers:
  Authorization: Bearer <GITHUB_PAT>
  Accept: application/vnd.github+json
Body:
{
  "event_type": "sharepoint-file-updated",
  "client_payload": {
    "tower": "FPR",
    "capability": "DS-001",
    "file_name": "CurrentFlows.xlsx",
    "sp_path": "Architecture/SAD/FPR/DS-001/input/data/CurrentFlows.xlsx"
  }
}
```

Step 3: Manual Reverse Sync (Alternative)

If Power Automate is not configured, you can trigger the reverse sync manually:

> **Actions → SharePoint Reverse Sync → Run workflow**

> - Tower shortcode: `FPR`
> - Capability ID: `DS-001`
> - File name: `CurrentFlows.xlsx`

This downloads the specified file from SharePoint and commits it to the repo. The push then triggers the forward generation workflow automatically.

Sync Sequence Summary

Step	What Happens	Triggered By
1	Architect edits Excel on SharePoint	Manual
2	Power Automate sends `repository_dispatch`	SharePoint file change trigger
3	`sharepoint-reverse-sync.yml` downloads file, commits	`repository_dispatch` event
4	`generate-architecture.yml` regenerates docs	Push to `main` (paths: `towers/**/input/**`)
5	Updated HTML/PDF/SVG uploaded to SharePoint	End of generation workflow
6	`deploy-pages.yml` deploys HTML to GitHub Pages	After generation workflow completes

Viewing HTML Outputs

GitHub does not render HTML files inline — it shows the source code instead. Three options for viewing the generated architecture HTML files:

Option 1: GitHub Pages (Recommended)

GitHub Pages serves the HTML files as a static website directly from the repository.

One-time setup on GitHub:

1. Go to `https://github.com/<OWNER>/IAO-Architecture/settings/pages`
2. Under **Build and deployment** → **Source**, select **GitHub Actions**
3. That's it — the `deploy-pages.yml` workflow handles the rest

How it works:

- Every time `generate-architecture.yml` completes successfully, it triggers `deploy-pages.yml`
- `deploy-pages.yml` copies all HTML and SVG outputs to a `_site/` folder and generates an index page
- GitHub Pages publishes the site at:

```
https://<username>.github.io/IAO-Architecture/
```

The index page lists every tower and capability with clickable links.

Direct capability link format:

```
https://<username>.github.io/IAO-Architecture/towers/<TOWER>/<ProcessGroup>/<CAP>/output/docs/systems-archit
```

> **Note:** If you already enabled Pages with "Deploy from a branch" (which creates a `static.yml` workflow), switch the source to **GitHub Actions** so that `deploy-pages.yml` is used instead. You can delete the `static.yml` workflow after switching.

Option 2: htmlpreview.github.io

For quick one-off viewing without any setup:

```
https://htmlpreview.github.io/?https://github.com/<OWNER>/IAO-Architecture/blob/main/towers/FPR/DS%20Provide
```

> **Caveat:** Spaces in folder names must be URL-encoded (`%20`). Mermaid diagrams may not render if htmlpreview blocks CDN scripts.

Option 3: Local

```
# Open a specific HTML file in your default browser (Windows)
start "towers\FPR\DS Provide Decision Support\DS-020\output\docs\systems-architecture\DS-020-Architecture.ht
```

SharePoint Input Folder Sync

For the reverse sync to work, the **input Excel files** must also be present on SharePoint. Upload the initial set manually or extend `sync_sharepoint.py` to push `input/data/` folders:

```
# The forward sync currently uploads output files only.
# Input xlsx files are synced via the reverse-sync workflow (SharePoint → GitHub).
# To bootstrap: manually upload the input/data/ folders to SharePoint,
# matching the structure: Architecture/SAD/<TOWER>/input/data/*.xlsx
```

The architect's workflow:

1. Navigate to `Shared Documents > Architecture/SAD > FPR > input/data/`
2. Open and edit `CurrentFlows.xlsx` (or `FutureFlows.xlsx` , `R3_CurrentFlows.xlsx` , `R3_FutureFlows.xlsx`)
3. Save — Power Automate detects the change and triggers the pipeline
4. Updated HTML/PDF appears in the output folder within minutes

Scheduling Options

The `generate-architecture.yml` workflow runs weekly by default. To change the schedule:

Frequency	Cron Expression	Description
Daily	`0 6 * * *`	Every day at 06:00 UTC
Weekly (default)	`0 6 * * 1`	Every Monday at 06:00 UTC
Bi-weekly	`0 6 1,15 * *`	1st and 15th of each month
Monthly	`0 6 1 * *`	First day of each month

Edit the `cron` line in `.github/workflows/generate-architecture.yml`:

```yaml
on:
  schedule:
    - cron: "0 6 * * *"   # Change to desired frequency
```

Or trigger an ad-hoc run from **Actions** → **Generate Architecture Docs** → **Run workflow**.

License

Internal use only.